Aula 17 – Paginação, Segmentação e Dispositivos de Entrada e Saída

Norton Trevisan Roman

23 de outubro de 2017

Paginação

- Endereços de disco usados para armazenar páginas que não estão na memória não fazem parte da tabela de páginas
  - A tabela contém apenas a informação que o *hardware* precisa para traduzir endereço virtual em físico
- Onde então colocar as páginas no disco, quando retiradas da memória?
  - E como saber onde estão? Como gerenciar isso?

Paginação – Partição de Swap

- A solução mais simples é ter uma partição especial de swap
  - Solução do Unix e Linux
  - Não possui um sistema de arquivos normal
    - São usados números de blocos relativos ao início da partição
- Quando o SO inicia, a partição está vazia
  - Representada na memória como uma única entrada contendo sua localização e tamanho
  - À medida em que processos são iniciados, o SO reserva um pedaço da área de swap do tamanho do processo
    - Quando terminam, o espaço é liberado

Paginação – Partição de Swap

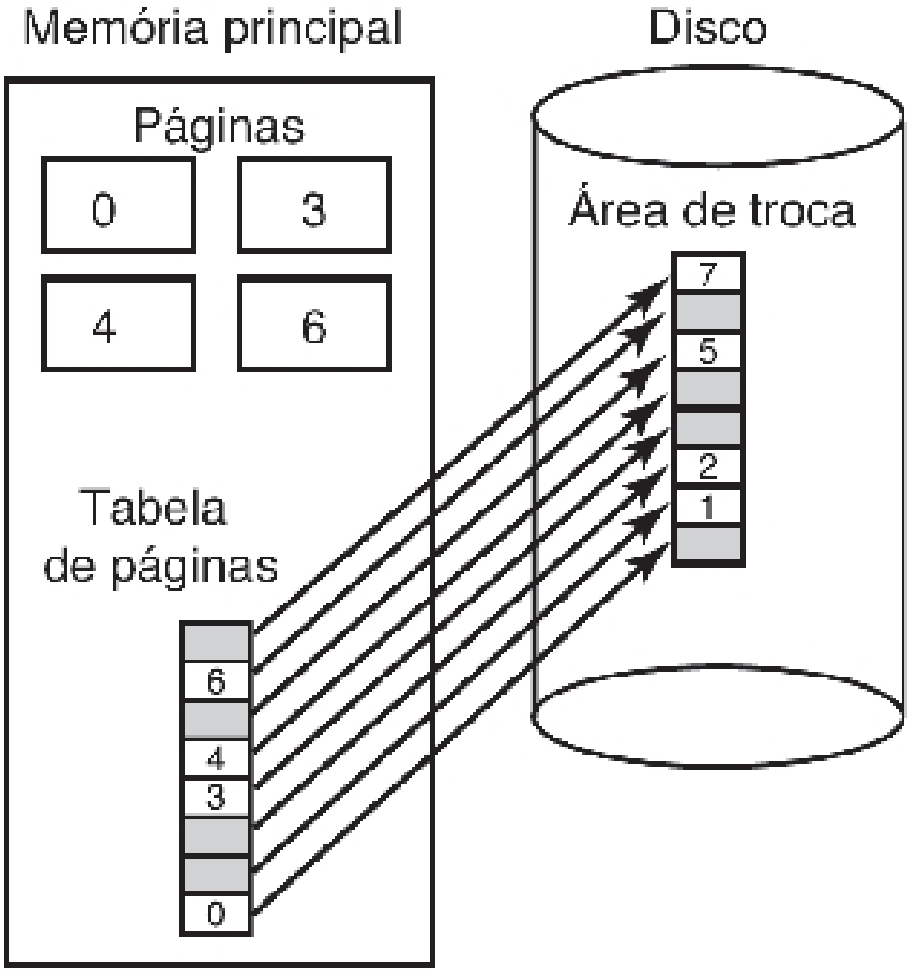
- A área de troca é gerenciada como uma lista de espaços disponíveis
  - Há algoritmos melhores, mas que não serão discutidos
- Associado a cada processo está o endereço no disco de sua área de swap
  - Mantido na tabela de processos
  - Cálculo do endereço para escrever uma página:
    - Adicione o endereço-base da página em seu espaço de endereçamento virtual ao endereço inicial da área de swap associada ao processo

Paginação – Partição de Swap

- Problema: antes de um processo iniciar, a área de swap deve ser inicializada
  - Possibilidade A – Assim que o processo é criado, ele é copiado todo para sua área de troca no disco, sendo carregado para memória quando necessário
    - Alternativamente, podemos copiá-lo todo para a memória, deixando que suas páginas sejam derrubadas se necessário
    - Problema: processos podem aumentar de tamanho enquanto executam (pilha e dados)
    - Solução: reservar áreas de troca diferentes para código, dados e pilha, permitindo que elas consistam de mais de um bloco no disco

Paginação – Partição de Swap

- Possibilidade A:
  - Basta saber o endereço do início da área de swap do processo → As páginas são espelhadas no disco
    - A área de swap é tão grande quanto o espaço de endereçamento virtual do processo
  - Área de troca (swap) estática

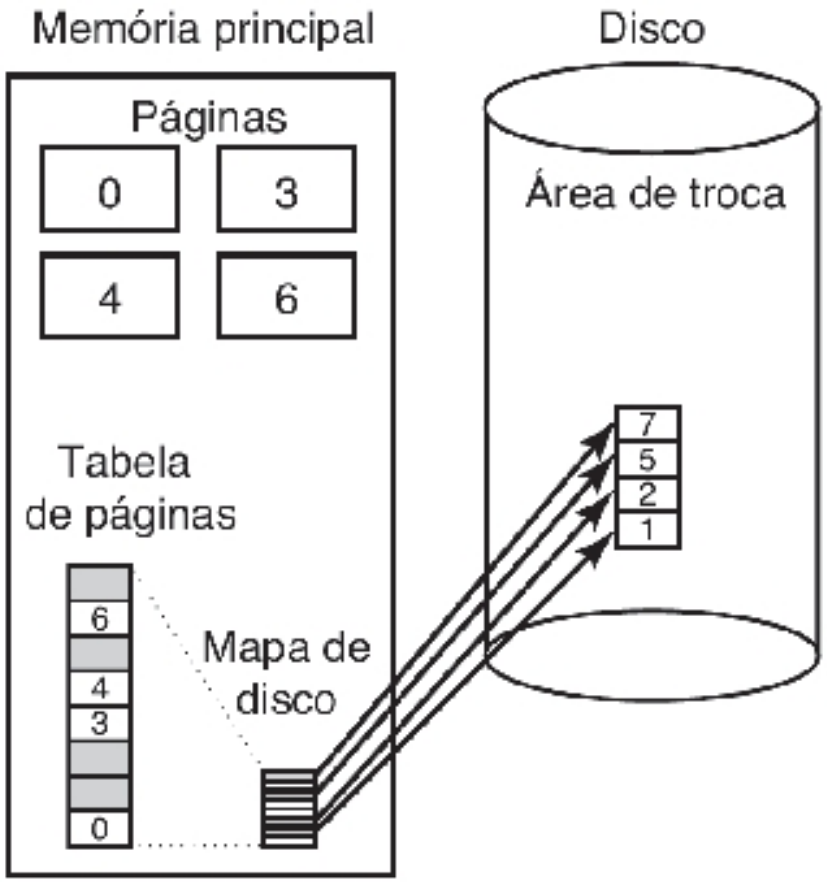


Paginação – Partição de Swap

- Possibilidade B – Nada é alocado antecipadamente.
  - Espaço é alocado em disco quando a página for enviada para lá e desalocado quando o processo termina
    - Alocação feita a cada página
    - Páginas "somente leitura" e as nunca modificadas são re-lidas dos arquivos originais, não ficando no swap
  - Assim, o processo na RAM não fica "amarrado" a uma área de swap específica
  - Desvantagem: precisamos manter na memória o endereço de cada página armazenada em disco
    - Deve haver uma tabela para cada processo dizendo onde cada página está no disco (se estiver lá)
    - Antes, bastava saber onde o processo iniciava no disco

Paginação – Partição de Swap

- Possibilidade B:
  - As páginas não têm endereço fixo
    - Quando uma página é derrubada, uma vazia em disco é escolhida e o mapa atualizado
    - Páginas na memória não têm cópia em disco (a não ser que tenha sido modificada em algum momento da execução)
  - Área de troca dinâmica



Paginação – Swap em Arquivo

E quando não podemos ter partição de swap?

- Podemos usar um ou mais arquivos grandes, pré-alocados, dentro do sistema de arquivos normal
  - Usado no Windows e em celulares (Linux também suporta)
- Windows 10:
  - C:\pagefile.sys

Paginação – Swap em Arquivo

- Permite otimizações:
  - Uma vez que o texto do programa veio de algum executável, podemos usar o executável como área de swap para o texto
  - O mesmo esquema pode ser usado com bibliotecas compartilhadas (shared libraries)
  - Lembre que o código é em geral read-only (não precisa ser devolvido da memória para o disco)
- Problemáticas em caso de falta de energia $\rightarrow$ o sistema de arquivos pode ficar inconsistente

Segmentação

Segmentação

- A memória virtual é unidimensional
  - Endereços virtuais vão de 0 a um máximo
- Considere agora o problema de compartilhamento de partes do código
  - Como fazer? Compartilhar cada página necessária? E quando o código está dentro da página, mas não a ocupa inteira?
- E se algum trecho precisar crescer?
  - Como fazer? Deslocar as demais páginas?

Segmentação

Solução: Segmentos

- Espaços de endereçamento independentes e separados
- Sequência linear de endereços, de 0 a um máximo
  - Seu tamanho está entre 0 e esse máximo
- Podem ter tamanhos diferentes, e o tamanho pode variar durante a execução
  - Ainda assim, até um máximo → pode "encher"
- Especificação de endereço feita em 2 partes:
  - Número do segmento e o endereço dentro do segmento

Segmentação

- Segmentos são entidades lógicas, e assim são vistos pelo programador
  - Podem conter uma rotina, um arranjo, uma coleção de variáveis, pilha etc
    - Ex: alguns object files (.o) são segmentados conforme o tipo de dado
  - Normalmente não contêm misturas de tipos
  - O usuário tem a ilusão de que todos os segmentos estão na memória principal, o tempo todo
- Diferem-se da paginação em que os segmentos não possuem tamanho fixo, enquanto que páginas possuem

Segmentação

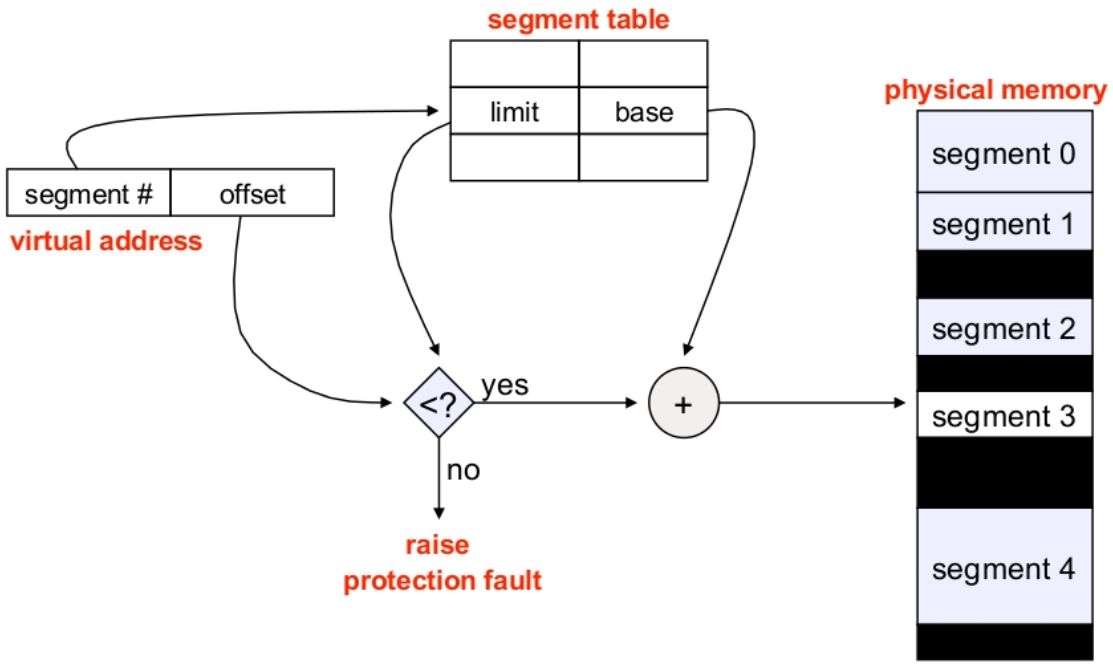
Implementação

- Tabelas de segmentos com n linhas, cada qual apontando para um segmento de memória
  - Vários espaços de endereçamento
- Alocação de segmentos segue os algoritmos já vistos
  - FIRST-FIT, BEST-FIT, NEXT-FIT, WORST-FIT, QUICK- FIT
- MMU também é utilizada para mapeamento entre os endereços lógicos e físicos
  - Tabela de segmentos informa qual o endereço da memória física do segmento e seu tamanho

Segmentação – Implementação

Suporte do hardware:

- Múltiplos pares base/limite, um por segmento
  - Armazenados na tabela de segmentos
  - O número do segmento é usado como índice na tabela
  - Há um número fixo desses pares



Segmentação – Implementação

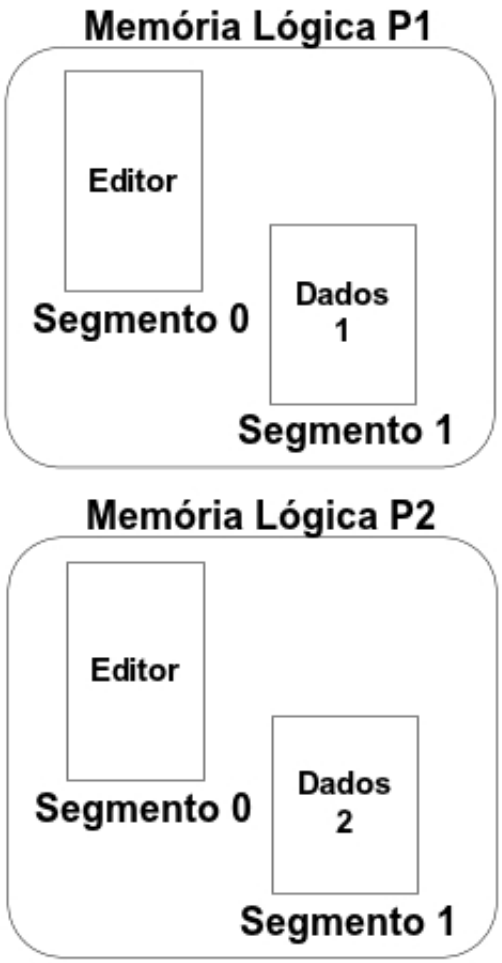
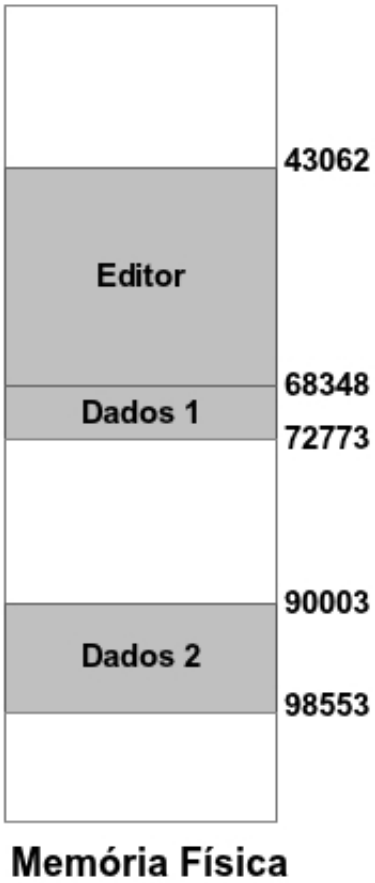


Tabela de Segmentos P1

	Limite	Base
0	25286	43062
1	4425	68348

Tabela de Segmentos P2

	Limite	Base
0	25286	43062
1	8850	90003



Segmentação: Problemas Encontrados

- Embora haja espaço na memória, não há espaço contíguo:
  - Política de realocação: um ou mais segmentos são realocados para abrir espaço contínuo
  - Política de compactação: todos os espaços são compactados
  - Política de bloqueio: fila de espera
  - Política de troca: substituição de segmentos
- Não possui fragmentação interna, embora sofra de fragmentação externa

Segmentação: Vantagens

- Facilita proteção dos dados
  - Diferentes segmentos podem possuir diferentes tipos de proteção
- Facilita compartilhamento de procedimentos e dados entre processos (Ex: bibliotecas compartilhadas)
  - Ex: Suponha que cada rotina ocupe um segmento
    - A rotina inicia no endereço (n,0) $\rightarrow$ (segmento, deslocamento)
  - Se mudamos a rotina em (5,0) e a recompilamos, nada mais tem que ser mudado (não mudamos nenhum outro segmento)
    - Em um esquema unidimensional, ao aumentarmos uma rotina, podemos mudar o endereço inicial da outra (podemos ter que rever todo o código)

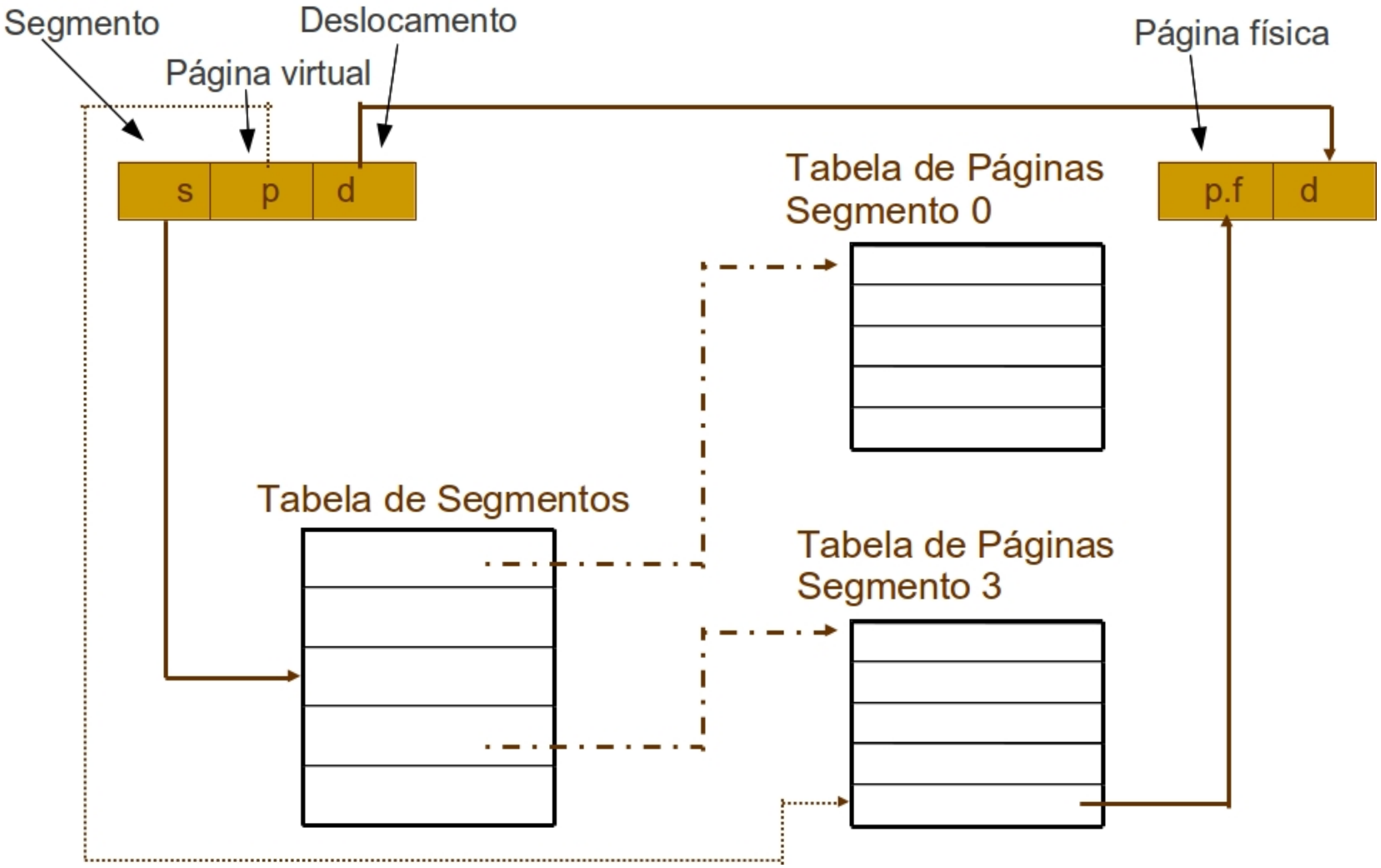
Segmentação Paginada

- Segmentos grandes podem não caber na memória
  - Devemos paginá-los (Ex: pentium)
- Espaço lógico é formado por segmentos
  - Cada segmento é dividido em páginas lógicas
  - Cada segmento possui uma tabela de páginas
    - Usada para mapear o endereço de página lógica do segmento em endereço de página física
  - No endereçamento, a tabela de segmentos indica, para cada segmento, onde sua respectiva tabela de páginas está
    - Semelhante à paginação multinível

Segmentação Paginada

- Ao referenciar um endereço de memória:
  - Endereço virtual: Número do segmento + Número da página + Deslocamento
  - Verifica se a página do segmento está na memória
    - Bit de residência
    - Senão estiver → page fault
  - Obtém, na tabela de segmentos, o endereço da tabela de páginas para aquele segmento
    - Usa o endereço da página virtual como offset nessa tabela
  - Da tabela de páginas, obtém o endereço físico da moldura
    - Age como em paginação

Segmentação Paginada



Segmentação Paginada

Consideração	Paginação	Segmentação
O programador precisa saber que essa técnica está sendo usada?	Não	Sim
Há quantos espaços de endereçamento linear?	1	Muitos
O espaço de endereçamento total pode superar o tamanho da memória física?	Sim	Sim
Rotinas e dados podem ser distinguidos e protegidos separadamente?	Não	Sim
As tabelas cujo tamanho flutua podem ser facilmente acomodadas?	Não	Sim
O compartilhamento de rotinas entre os usuários é facilitado?	Não	Sim
Por que essa técnica foi inventada?	Para obter um grande espaço de endereçamento linear sem a necessidade de comprar mais memória física	Para permitir que programas e dados sejam divididos em espaços de endereçamento logicamente independentes e para auxiliar o compartilhamento e a proteção

Entrada e Saída

Entrada e Saída

- Funções específicas do S.O.:
  - Enviar sinais para os dispositivos
  - Atender interrupções
  - Gerenciar comandos aceitos e funcionalidades (serviços prestados)
  - Tratar possíveis erros
  - Prover interface entre os dispositivos e o sistema
- Geralmente, o código para tratamento da entrada e saída representa uma fração significativa do sistema operacional total

Entrada e Saída

Classificação quanto ao Tipo de Transferência

- Dispositivos de bloco (Block devices):
  - Armazenam informação em blocos de tamanho fixo, cada um com seu próprio endereço
    - Os tamanhos dos blocos variam de 512 B a 32 KB
  - Transferências são feitas com um ou mais blocos consecutivos
  - Cada bloco é lido ou escrito independentemente dos demais
  - Ex: HD, CD-ROM, Bastão USB
    - O sistema de arquivos, por exemplo, trata com dispositivos de bloco abstratos

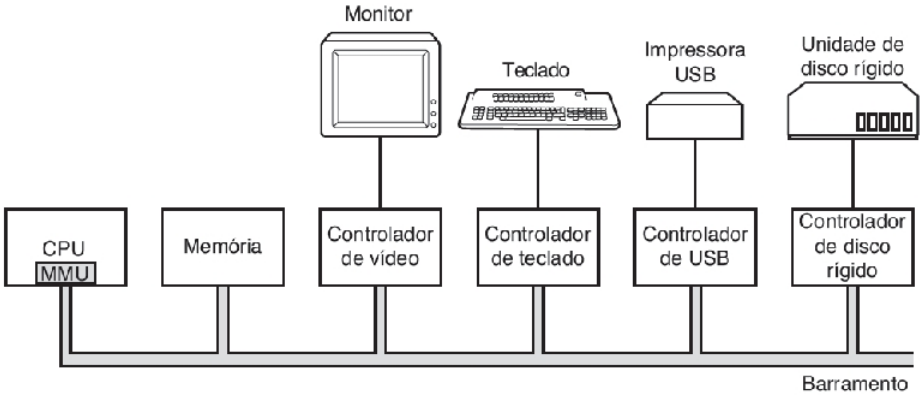
Entrada e Saída

Classificação quanto ao Tipo de Transferência

- Dispositivos de caractere (Character devices)
  - Enviam ou recebem um fluxo de caracteres, sem considerar qualquer estrutura de bloco
  - Não são endereçáveis e não possuem qualquer operação de posicionamento ("seek operation")
  - Ex: impressoras, interfaces de rede, mouses
- Este esquema de classificação não é perfeito
  - O timer (clock) não se encaixa em nenhuma categoria dessas

E/S: Princípios de Hardware

- O S.O. sempre trata com a controladora, não com os dispositivos
  - A Comunicação entre CPU e controladoras é feita através de barramentos comuns (interface de alto nível)
  - Interface entre controladora e dispositivo: baixo nível
  - Mainframes: múltiplos barramentos e processadores especializados em E/S (canais de E/S).



Princípios de Hardware: Controladores

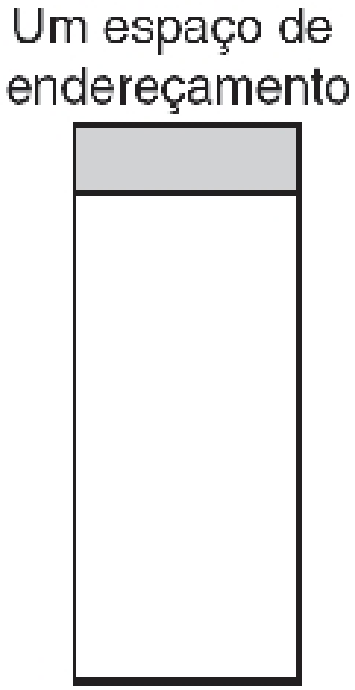
- Cada controladora possui registradores, usados para comunicação com a CPU
  - Ao escrever nesses registradores, o SO pode comandar o dispositivo
    - Executa E/S escrevendo comandos (e seus parâmetros, se existirem) nesses registradores
    - Quando um comando é aceito, a CPU pode deixar que a controladora trabalhe sozinho, indo executar outra tarefa.
    - Quando o dispositivo termina, avisa a CPU através de uma interrupção.
  - Ao ler desses registradores, o SO pode saber o estado do dispositivo etc

Princípios de Hardware: Controladores

- Muitos dispositivos possuem também um buffer, que o SO pode ler (ou escrever)
  - Ex: memória de vídeo
- Como a CPU se comunica com esses registradores e buffers de dados?
  - Necessidade de associação de endereços a estes dispositivos

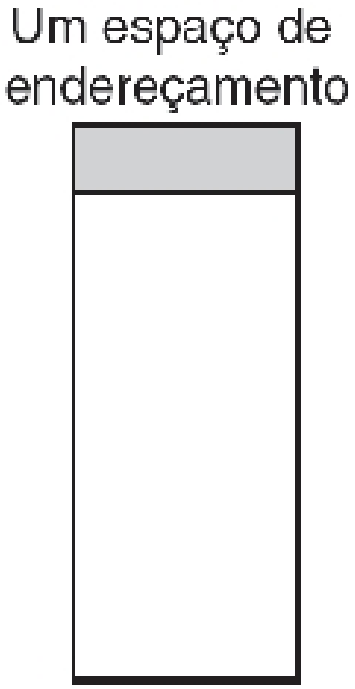
Comunicação CPU – Controladores

- Existem 2 formas de interpretação desses endereços pelo módulo de E/S quando o barramento é compartilhado:
  - Mapeado em memória ou em E/S
  - Ambos implementados pelo hardware
- Mapeado em Memória:
  - O módulo de E/S opera dentro do espaço de endereçamento de memória, usando um conjunto de endereços reservados (registradores são tratados como posições de memórias)



Comunicação CPU – Controladores

- Mapeado em Memória:
  - Mapeia todos os registradores de controle no espaço de memória
    - Usa o mesmo barramento para endereçar memória e dispositivos
  - Cada registrador de controle recebe um endereço de memória único
    - Geralmente o topo do espaço de endereços
    - Nenhuma memória é associada a esses endereços
    - Quando um endereço de memória correspondente a um registrador é usado, a CPU faz o desvio

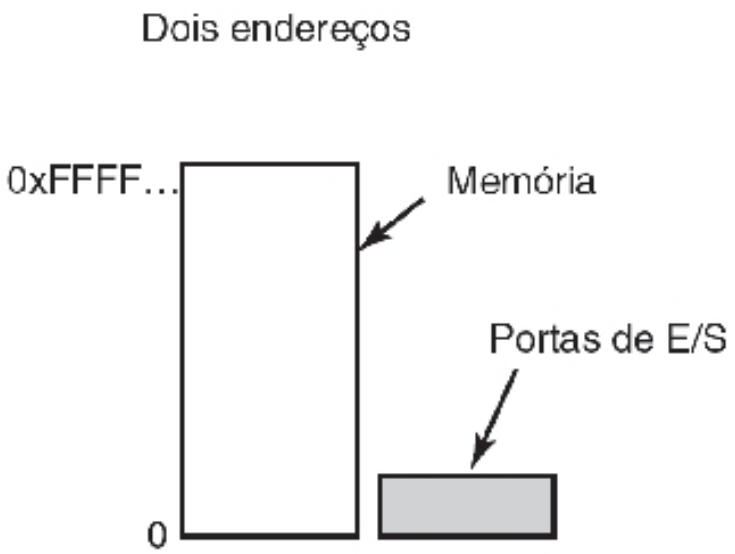


Comunicação CPU – Controladores

- Mapeado em Memória – Vantagens:
  - Não há a necessidade de instruções especiais de E/S
    - A interação se dá como se fosse uma posição de memória comum
  - Para impedir que determinados usuários acessem o dispositivo, basta não mapear as páginas em seu espaço de endereçamento virtual
- Desvantagens:
  - Consome parte do espaço de endereçamento
  - Permitir o uso da cache para os registradores de controle pode ser desastroso
    - O hardware deve poder desabilitar a cache seletivamente (em cada página, por exemplo)

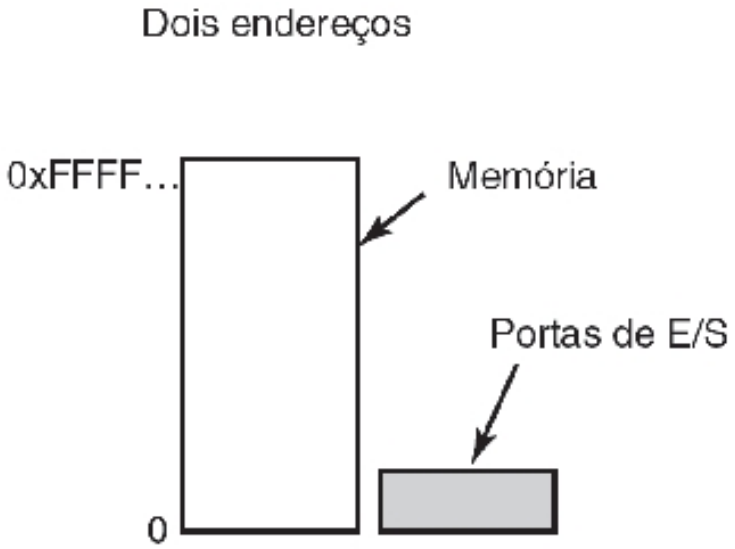
Comunicação CPU – Controladores

- Mapeado em E/S ou E/S isolada (I/O Ports):
  - Cada registrador de controle recebe um número de porta de E/S (inteiro de 8 ou 16 bits)
  - Acessada via instruções especiais (Intel), usadas apenas pelo SO
    - IN REG, PORT → leia do registrador de controle PORT e armazene no registrador REG da CPU
    - OUT PORT,REG → caminho inverso de IN



Comunicação CPU – Controladores

- Mapeado em E/S:
  - Espaços de endereços diferentes para memória e E/S
    - Existe um espaço de endereçamento independente para os dispositivos de E/S
    - IN RO,4 → lê da porta de E/S 4 para RO
    - MOV RO,4 → lê do endereço 4 da RAM para RO

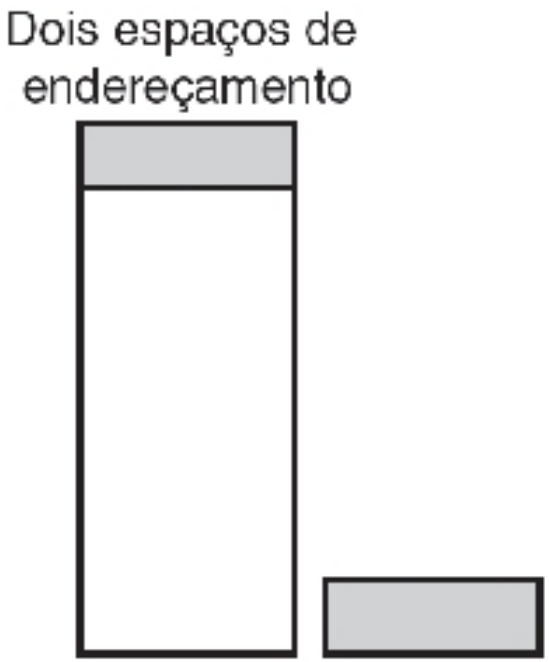


Comunicação CPU – Controladores

- Mapeado em E/S – Vantagens:
  - Não gasta espaço de endereçamento
  - Não sofre com o uso da Cache
- Desvantagens:
  - Requer instruções especiais para acesso
    - Drivers necessariamente tem que usar algum assembly para acessar as instruções

Comunicação CPU – Controladores

- Sistemas híbridos
  - Buffers de dados de E/S mapeados na memória
  - Portas de E/S para os registradores de controle
  - Ex: Pentium
    - Portas de E/S de 0 a 64K-1
    - Endereços de 640K a 1M-1 reservados aos buffers (experimente ver o gerenciador de recursos (memória) do Windows ou o resultado de *free* do Linux)



Comunicação CPU – Funcionamento

- Quando a CPU quer ler uma palavra (E/S ou RAM):
  - Coloca o endereço nas linhas de endereço do barramento
  - Emite um sinal READ sobre uma linha de controle do barramento
  - Em uma segunda linha de sinal informa se o espaço é de E/S ou de memória

E/S – Princípios de Software: Objetivos

- Independência de dispositivo
  - O programador não deve se preocupar com o tipo de dispositivo (CD, HD etc)
  - Cabe ao SO cuidar das idiossincrasias
- Nomenclatura uniforme
  - O nome de um dispositivo deve ser um string ou inteiro, independente do dispositivo
- Tratamento de erro
  - Erros deveriam ser tratados o mais próximo do hardware possível, sem que o usuário tome conhecimento
  - Ex: repetindo a operação

E/S – Princípios de Software

Modos de execução de E/S:

- Os dispositivos de E/S podem operar de 3 modos básicos:
  - E/S programada
  - E/S via Interrupções
  - E/S via Acesso Direto à Memória
- O que distingue as três formas é a participação da CPU e a utilização das interrupções

E/S – Princípios de Software

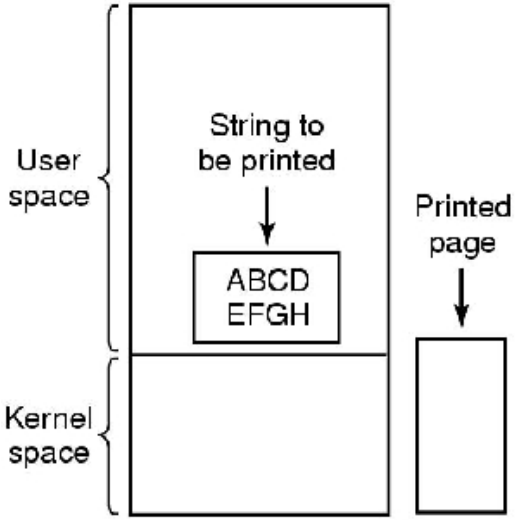
E/S programada

- Forma mais simples de E/S → tudo é feito pela CPU
- Os dados são trocados entre a CPU e o dispositivo
- A CPU executa um programa que:
  - Verifica o estado do dispositivo de E/S, preparando-o para a operação
  - Se necessário, envia o comando que deve ser executado; aguardando o resultado deste, para então efetuar a transferência entre o dispositivo de E/S e algum registrador da CPU

E/S – Princípios de Software

E/S programada

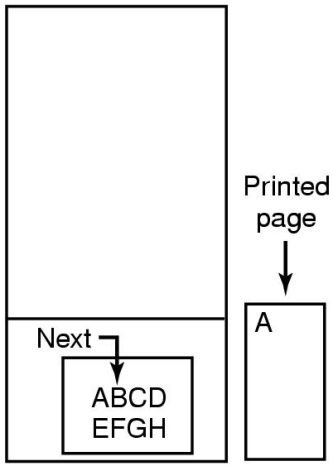
- Ex: imprimir "ABCDEFGH" na impressora
  - Monta-se o string em um buffer, no espaço do usuário
  - O processo então detém a impressora (via chamada de sistema – open)
  - Uma vez obtida a impressora, o processo diz ao SO para imprimir (write – chamada ao sistema)
  - O SO copia o string a um arranjo no espaço do kernel



E/S – Princípios de Software

E/S programada

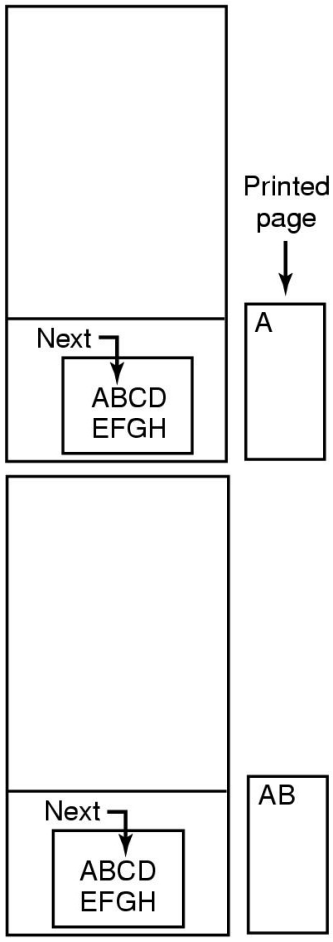
- Assim que a impressora é liberada, o SO copia o primeiro caractere ao registrador de dados da impressora
- O primeiro caractere é impresso



E/S – Princípios de Software

E/S programada

- Assim que a impressora é liberada, o SO copia o primeiro caractere ao registrador de dados da impressora
- O primeiro caractere é impresso
- O SO marca o próximo caractere, verifica novamente se a impressora está pronta (via registrador de status) para receber outro
- Se pronta, o novo caractere é enviado
- Continua até imprimir todo o string
- Só então o controle volta ao processo



E/S – Princípios de Software

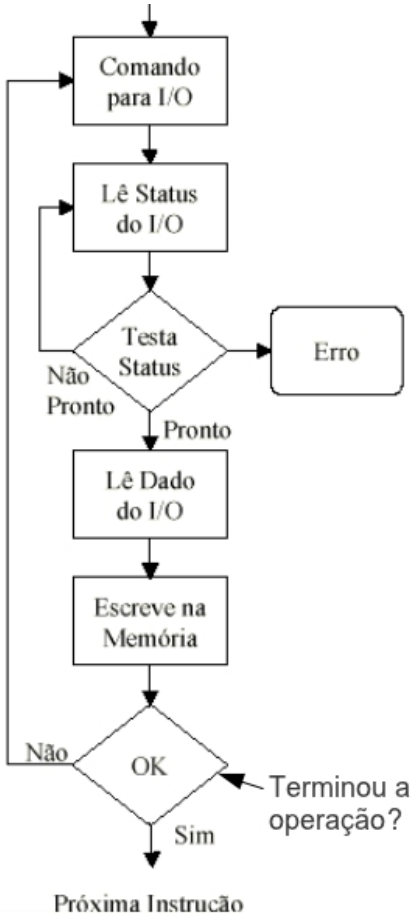
E/S programada

- Desvantagem:
  - CPU é ocupada o tempo todo até que a E/S seja feita
  - CPU continuamente verifica se o dispositivo está pronto para aceitar outro caractere → espera ocupada (também chamado de polling)

E/S – Princípios de Software

E/S programada

- Desvantagem:
  - CPU é ocupada o tempo todo até que a E/S seja feita
  - CPU continuamente verifica se o dispositivo está pronto para aceitar outro caractere → espera ocupada (também chamado de polling)
- Outro exemplo: Leitura de E/S
  - Escrita é semelhante



Referências Adicionais

- https://www.kernel.org/doc/gorman/html/understand/understand014.html
- http://en.wikipedia.org/wiki/Memory-mapped_I/O
- http://docstore.mik.ua/orelly/linux/run/ch06_02.htm